

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA
ESCUELA PROFESIONAL DE INGENIERÍA DE TELECOMUNICACIONES



TÍTULO:

“Implementación Física, Validación Experimental y Análisis de Métricas del Sistema
Controlador de Teclado Matricial para Comandos AT”

INTEGRANTES:

Chuzon Wickham, Renzo Felipe

Rivera Flores, Jorge David

Rodriguez Vasquez, Andrea Alexandra

I. OBJETIVOS

Este documento tiene como objetivo evidenciar el funcionamiento integral del sistema digital propuesto, detallando la interacción entre la capa de hardware, el puente de comunicación y la capa de aplicación. La FPGA MAX 10 adquiere las señales analógicas del teclado matricial 4x4, las procesa mediante una FSM y un decodificador, y las muestra en los displays de 7 segmentos físicos. Simultáneamente, la señal es enviada al Arduino y transmitida al entorno Python para la actuación final sobre la laptop. Asimismo, se presenta la validación de los decodificadores y el análisis de métricas experimentales para el proyecto.

II. ARQUITECTURA FINAL DEL SISTEMA

Capa de Hardware (FPGA MAX 10):

El núcleo del procesamiento a bajo nivel se realiza en la FPGA Terasic DE10-Lite. Este módulo cuenta con un reloj maestro de 50 MHz adaptado mediante divisores de frecuencia (Clock Enable a 1 kHz) para el escaneo del teclado matricial 4x4. Utiliza una Máquina de Estados Finitos (FSM) y un decodificador de hardware para interpretar las coordenadas de las teclas, las cuales se visualizan en tiempo real en los displays de 7 segmentos físicos de la placa.

Módulo de Interfaz y Puente de comunicación (Arduino):

Para lograr la comunicación entre la lógica del FPGA y la laptop, se implementó un microcontrolador Arduino actuando como Bridge Serial-USB. Este módulo recibe las tramas digitales provenientes de los pines GPIO de la FPGA y las adapta al estándar serie-USB para que puedan ser leídas por el puerto COM de la laptop, garantizando la integridad de los comandos AT transmitidos.

Capa de Aplicación y Actuación (Python/Laptop):

Los datos enrutados por el Arduino son recibidos por un script en Python. Esta capa de software cuenta con una interfaz gráfica que actúa como monitor del sistema. Dependiendo del comando recibido, el sistema no solo actualiza la interfaz visual, sino que ejecuta acciones a nivel del sistema operativo de la laptop conectada.

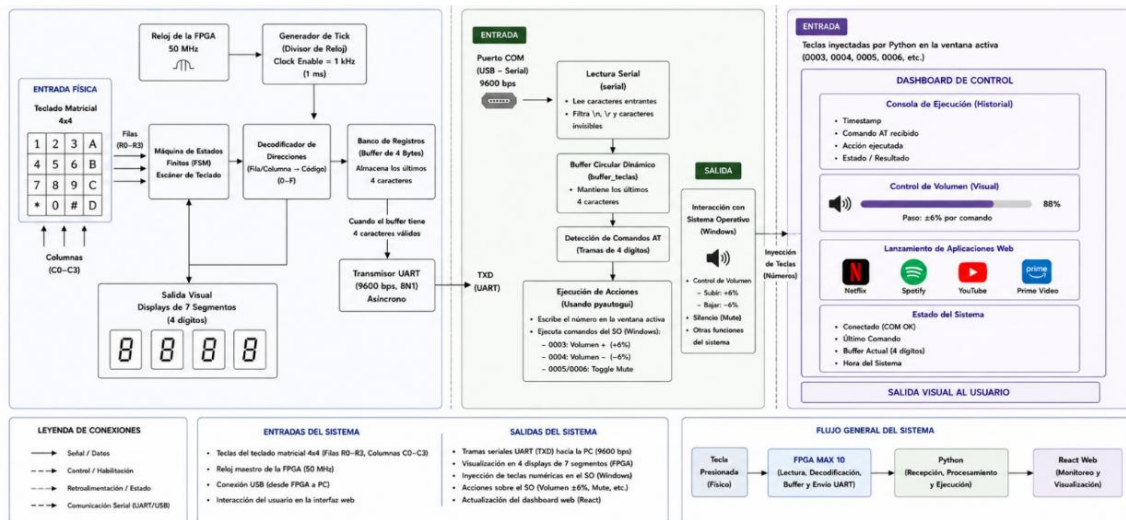


Figura 1.- Diagrama de bloques del flujo general del sistema.

ANÁLISIS DE MÉTRICAS EXPERIMENTALES

Métrica Evaluada	Descripción	Valor teórico	Valor real	Variación
Latencia de comando (ms)	Tiempo transcurrido desde la presión física de la tecla hasta la llegada al puerto UART.	18.16ms	25ms	6.84ms
Frecuencia de escaneo FSM (Hz)	Velocidad a la que la máquina de estados completa un ciclo completo del barrido de las 4 filas.	1000 Hz	985Hz	1.5%
Ocupación de recursos	Cantidad de memoria o registros lógicos utilizados.	310 elementos lógicos / 95 registros.	283 elementos lógicos / 84 registros / 40 pines.	Optimización

Tabla 1.- Tabla donde se registran las métricas evaluadas y su comparación con el modelo teórico.

controlador_teclado_AT.v

Compilation Report - controlador_teclado_AT

Table of Contents

Flow Summary

Flow Status: Successful - Mon Jun 26 21:06:49 2024

Quartus Prime Version: 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

Revision Name: controlador_teclado_AT

Top-level Entity Name: controlador_teclado_AT

Family: MAX 10

Device: 10M50DAF484C7G

Timing Models: Final

Total logic elements: 283

Total registers: 84

Total pins: 40

Total virtual pins: 0

Total memory bits: 0

Embedded Multiplier 9-bit elements: 0

Total PLLs: 0

UFM blocks: 0

ADC blocks: 0

Figura 2.- Reporte de compilación final obtenido en Quartus Prime.

III. EVIDENCIAS DE FUNCIONALIDAD OPERATIVA COMPLETA

Se evidencia el hardware ensamblado operando. El sistema escanea continuamente el teclado matricial utilizando una Máquina de Estados Finitos. Al detectar una pulsación, la lógica combinacional actúa como un decodificador, traduciendo la coordenada en un valor binario representativo.

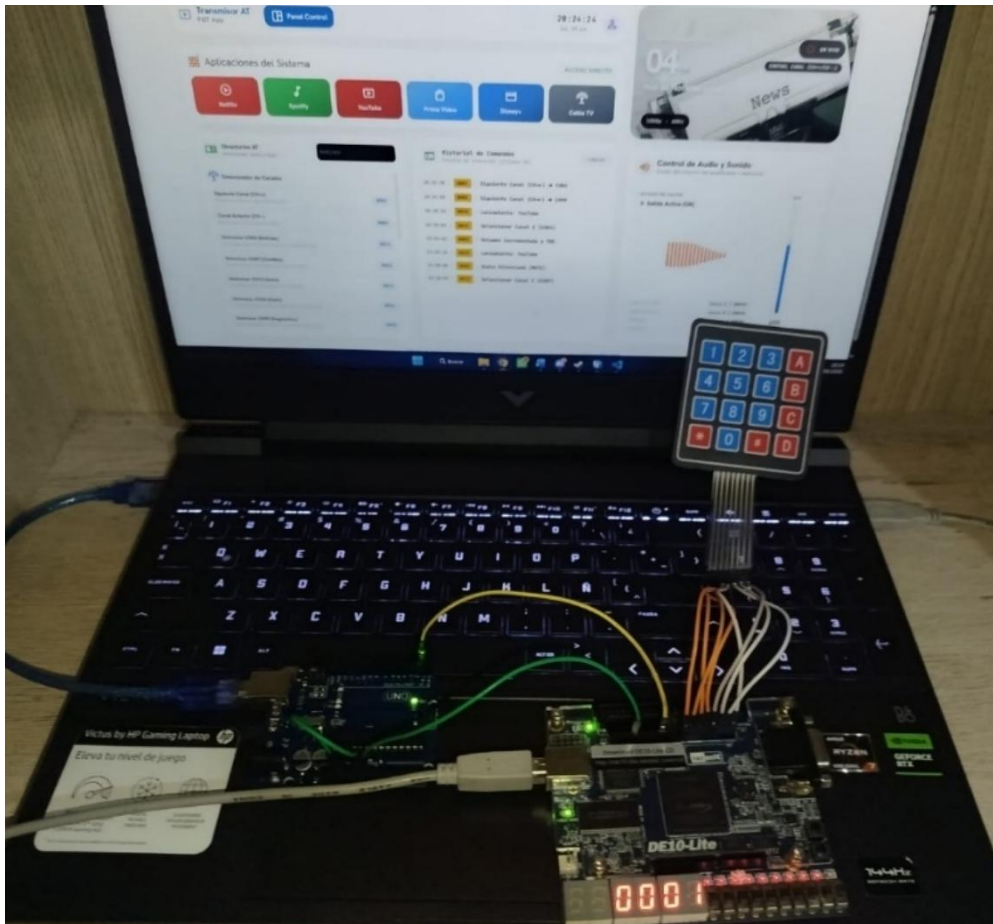


Figura 3.- Conexión física del teclado matricial 4x4 a los pines de entrada del FPGA MAX 10, además de la conexión del Arduino y la laptop que mostrará la interfaz.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Renzo W\Documents\V\Circ. Digitales\Proyecto teoría\Programa\transmisor-at---pqt-app> & "
C:\Users\Renzo W\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Renzo W/Documents/V/Ci
rc. Digitales/Proyecto teoría/Programa/transmisor-at---pqt-app/puente.py"

Memoria: 'A001'
Memoria: '001A'
Memoria: '01A0'
Memoria: '1A00'
Memoria: 'A000'
Memoria: '0000'
Memoria: '0000'
Memoria: '0000'
Memoria: '0001'
```

Figura 4.- Consola de depuración donde se observa la recepción y almacenamiento correcto de las tramas de los comandos AT en la memoria del sistema.

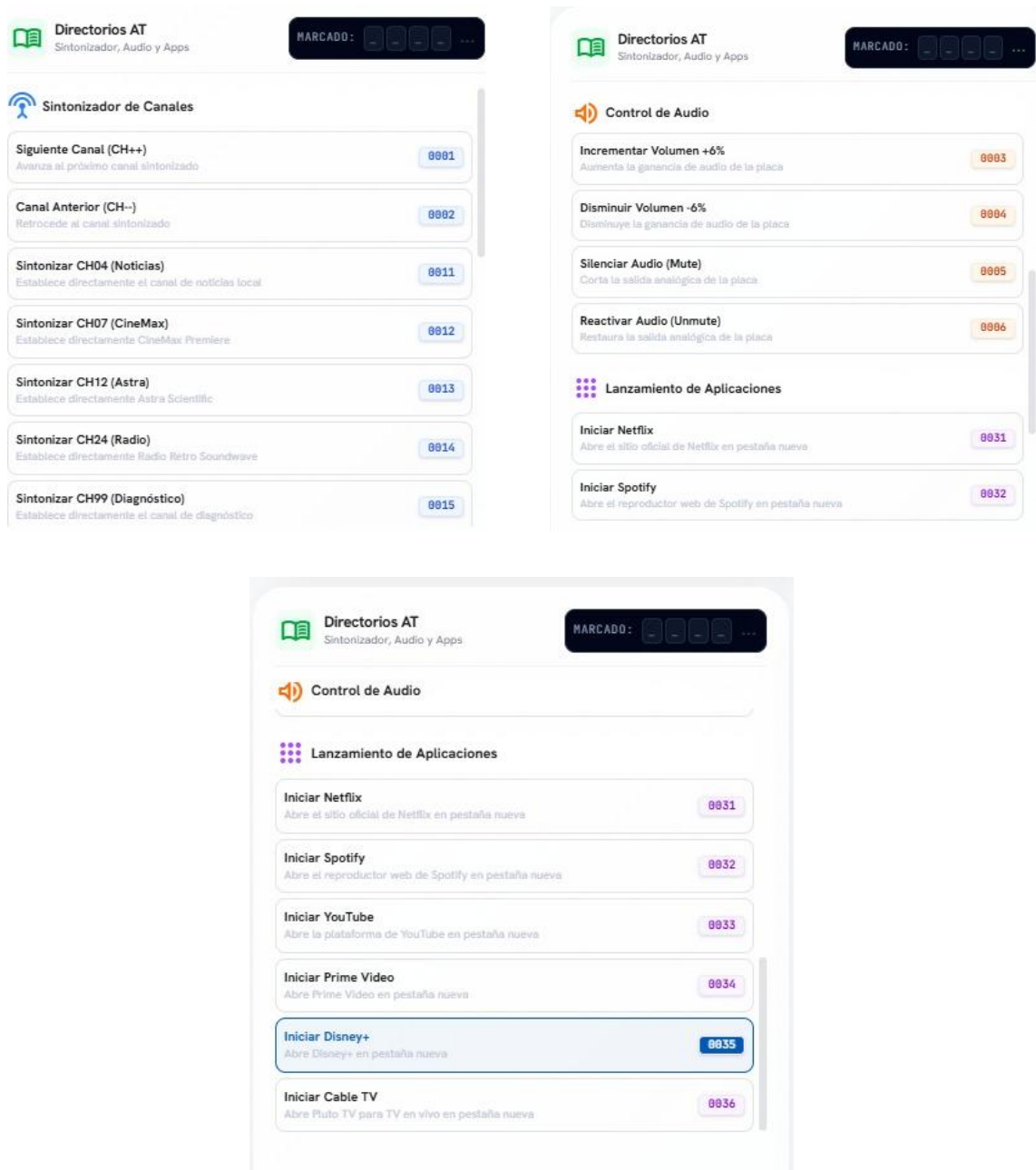


Figura 5.- Panel con el directorio de comandos disponibles.



Figura 6.- Vista del módulo del historial de comandos en la interfaz gráfica.

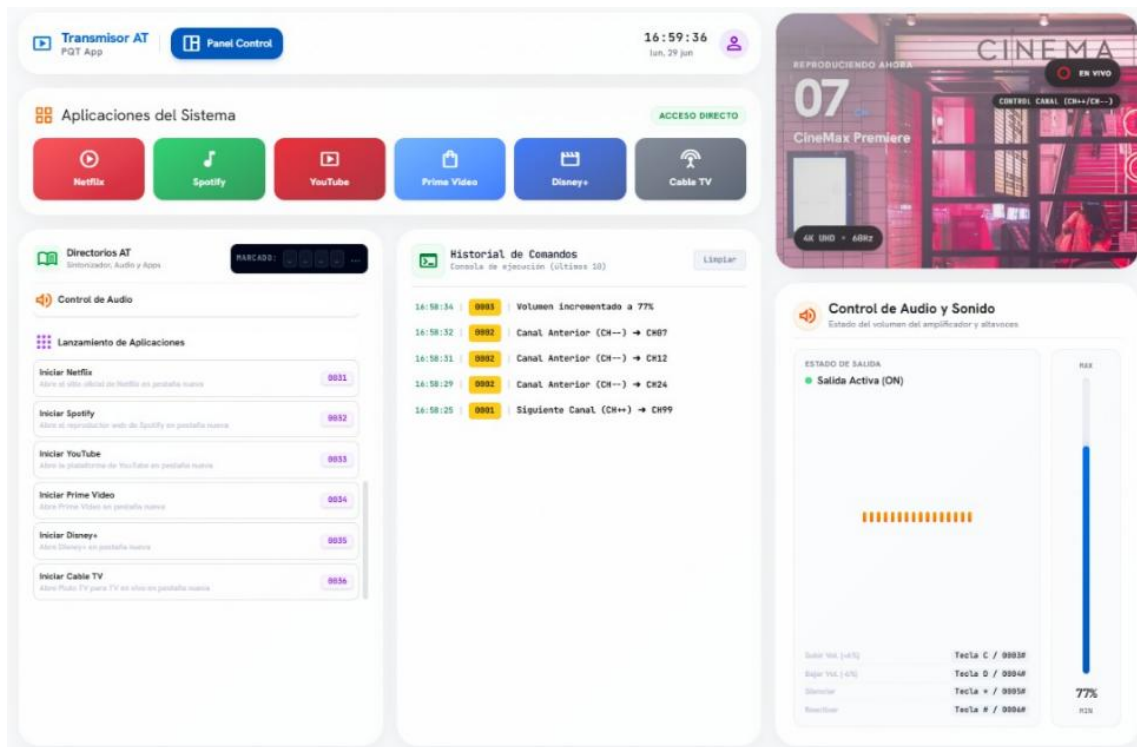


Figura 7.- Vista completa del dashboard de control desarrollado para el usuario.

IV. ANÁLISIS DE DESVIACIONES Y POSIBLES CAUSAS

- **Retraso en la respuesta de la laptop:** En la simulación, el procesamiento de datos es inmediato. Sin embargo, en el circuito físico notamos una pequeña demora de milisegundos desde que se presiona la tecla hasta que Python ejecuta la acción en la computadora. Esto se debe a la comunicación serial. El dato no va directo a la laptop, primero debe pasar de la FPGA al Arduino, y luego el Arduino lo envía por el cable USB a una velocidad limitada de transmisión.
- **Errores por cables sueltos:** Al realizar las pruebas moviendo el prototipo, notamos que a veces los comandos no se enviaban completos o la laptop no los reconocía. Debido a que el uso de cables jumper para conectar los pines de la FPGA con el Arduino introduce falsos contactos y ruido.
- **Diferencia de velocidad entre los dispositivos:** En ocasiones, si se presionaban las teclas de forma sumamente rápida, el Arduino no llegaba a registrar el comando enviado por la FPGA ya que la FPGA trabaja de forma paralela y a una velocidad alta de 50MHz, mientras que el Arduino procesa el código de forma secuencial. Si el Arduino está ocupado transmitiendo un dato anterior a la laptop, puede ignorar el nuevo dato enviado por la FPGA.

V. CONCLUSIONES

- El análisis del rendimiento mostró una diferencia importante entre las plataformas utilizadas. Mientras la FPGA procesa la información de manera rápida y paralela a 50 MHz, el microcontrolador actúa como una limitación debido a que trabaja de forma secuencial y depende de la comunicación serial. Por ello, el desempeño general del sistema está determinado por el componente más lento de la arquitectura.
- Las pruebas realizadas confirmaron que el filtro antirrebote es efectivo para eliminar las señales erróneas producidas por las teclas mecánicas. Sin embargo, se observaron algunos errores cuando las teclas eran presionadas de forma muy rápida o irregular.
- Durante las pruebas experimentales se observó que las conexiones realizadas mediante cables y pines pueden afectar la estabilidad de la señal cuando se producen movimientos o falsos contactos. Aunque el diseño lógico funciona correctamente, es importante asegurar conexiones físicas firmes para garantizar un funcionamiento estable y evitar errores en la transmisión de los datos.
- El desarrollo del proyecto permitió comprobar que es posible integrar correctamente diferentes tecnologías en un solo sistema. La FPGA procesó las señales del teclado matricial, el microcontrolador actuó como puente de comunicación y la computadora recibió los datos para ejecutar las funciones multimedia, demostrando que la arquitectura propuesta funciona de manera confiable.